

Jitsu QA – Database Cheat Sheet

****JITSU QA TEAM****

****Database Cheat Sheet****

*PostgreSQL **&** MongoDB Quick Reference*

****📄 INSIDE THIS DOC****

- ✓ 15 essential SQL queries for daily QA work
- ✓ PostgreSQL queries – clients, regions, drivers, assignments
- ✓ MongoDB queries – sort centers, BG checks, dimensions
- ✓ Common pitfalls and tips

Prerequisite: StrongDM setup (Training Doc Section 11.0)

Version 1 – May 2026

****How to Use This Cheat Sheet****

This document gives you the most common database queries you'll need as a QA at Jitsu. Copy → modify → run.

****💡 Prerequisite – StrongDM Connected**** Before running any query, ensure StrongDM Desktop Client is open and the relevant resource (Staging PostgreSQL or Staging MongoDB) shows ♥ Connected. See Training Document Section 11.0 for setup.

****Tool of choice****

- PostgreSQL → DBeaver or pgAdmin
- MongoDB → MongoDB Compass or Studio 3T

****Connection settings (both DBs)****

- Host: localhost
- Port: shown next to the resource in StrongDM client (varies)
- Authentication: not required (SDM handles it)

****💡 Convention used in queries**** All examples use ID 49 (Hello Fresh – canonical Parent Client in Production) and region 'SF0' for illustration. Replace with your test client/region IDs when running.

****Part 1 – PostgreSQL Queries****

PostgreSQL is the primary relational database – stores clients, regions, assignments, drivers, shipments.

****Clients****

🔍 ****Q1. Find clients by name (test clients)****

Lookup test clients created during QA work – always use Test_ prefix

SELECT id, name, email, logistic_type, created_at

```
FROM clients
WHERE name ILIKE 'Test_%'
ORDER BY created_at DESC
LIMIT 20;
```

💡 *ILIKE = case-insensitive LIKE. Use this to clean up after testing.*

🔍 **Q2. Find a specific client by ID**

When you know the client_id (e.g. from a ticket)

```
SELECT id, name, email, logistic_type, purchase_id, created_at
FROM clients
WHERE id = 49;
```

💡 *purchase_id shows billing link. If different from id, it's a Brand-with-client_id.*

🔍 **Q3. List clients by service type**

Find all clients of a specific logistic type

```
SELECT id, name, logistic_type
FROM clients
WHERE logistic_type = 0    -- 0=NEXT_DAYS, 1=ON_DEMAND, 2=SPECIALTY, 3=FCTD
ORDER BY name
LIMIT 30;
```

💡 *Useful when testing flow specific to a client service type.*

Client Services **&**** Regions**

🔍 **Q4. Check client_service entries for a client**

See which regions a client is active in – must add 1 entry per region

```
SELECT cs.client_id, c.name AS client_name,
       cs.region_id, r.name AS region_name,
       cs.logistic_type
FROM client_service cs
JOIN clients c ON cs.client_id = c.id
JOIN regions r ON cs.region_id = r.id
WHERE cs.client_id = 49;
```

💡 *If empty, the client cannot submit shipments. Add entries via INSERT for each region.*

🔍 **Q5. List all regions with their client type**

Critical: 1 Region = 1 Client Type. Check region's type before assigning a client.

```
SELECT id, name, client_type, state
FROM regions
ORDER BY name;
```

💡 *client_type values: 0=NEXT_DAYS, 1=ON_DEMAND, 2=SPECIALTY, 3=FCTD.*

🔍 **Q6. Add a client_service entry (INSERT)**

Link a client to a region – must match the region's client type

```
INSERT INTO client_service (client_id, region_id, logistic_type)
VALUES (49, 5, 0);    -- Client 49, Region 5 (SFO), NEXT_DAYS
```

-- Verify after insert:

```
SELECT * FROM client_service WHERE client_id = 49 AND region_id = 5;
```

💡 *Run SELECT first to check the region's type matches the client's type.*

Drivers

🔍 **Q7. Find a driver by ID**

Lookup driver info – including type (IC/DSP/3P/Linehaul) and status

```
SELECT id, name, email, phone, driver_type, status, region_id
FROM drivers
WHERE id = 7873;
```

💡 *driver_type values: IC, DSP, 3P, LINEHAUL. status: ACTIVE, INACTIVE, BANNED.*

🔍 **Q8. Find test drivers (by email or name prefix)**

Find test drivers created during QA – always use Jitsu email

```
SELECT id, name, email, driver_type, status
FROM drivers
WHERE email ILIKE '%@jitsu.com'
      OR email ILIKE '%@gojitsu.com'
      OR name ILIKE 'Test_%'
ORDER BY id DESC
LIMIT 20;
```

💡 *Useful before cleaning up test drivers after E2E tests.*

Assignments *** Shipments**

🔍 **Q9. Find assignments for today**

List all assignments created today – useful for daily smoke checks

```
SELECT id, region_id, vehicle_type, shipment_count, status, created_at
FROM assignments
WHERE created_at >= CURRENT_DATE
ORDER BY created_at DESC
LIMIT 20;
```

💡 *Combine with region filter (AND region_id = 5) to narrow scope.*

🔍 **Q10. Count shipments by status today**

Quick health check on today's shipment distribution

```
SELECT status, COUNT(*) AS count
FROM shipments
WHERE created_at >= CURRENT_DATE
GROUP BY status
ORDER BY count DESC;
```

💡 *Common statuses: GEOCODED, ASSIGNED, PICKUP_SUCCEEDED, DROPOFF_SUCCEEDED, DROPOFF_FAILED.*

🔍 **Q11. Find a shipment by tracking code**

When customer/client gives a tracking code

```
SELECT id, tracking_code, status, client_id, assignment_id,  
       recipient_name, recipient_address, created_at  
FROM shipments  
WHERE tracking_code = 'JT123456789';
```

💡 *Always use exact match for tracking codes.*

Warehouses

🔍 **Q12. List warehouses by region**

Find which warehouses serve a region

```
SELECT id, name, type, region_id, address, status  
FROM warehouses  
WHERE region_id = 5;
```

💡 *type values: MAIN, MOBILE_HUB, CLIENT, DSP. Use this when setting up test data.*

Part 2 – MongoDB Queries

MongoDB stores configuration documents, registration records, and time-sensitive operational data.

Sort Center Configuration

🔍 **Q13. Check sort center config**

Find dimension measurement settings per sort center

```
// MongoDB shell or Compass  
db.sort_center.find(  
  { warehouse_id: 193 },  
  { warehouse_id: 1, enable_measure: 1, dimension_config: 1 }  
);
```

💡 *enable_measure controls whether the 'Measure' label flow is active. Used in Section 11.10 Dimensions test.*

Driver Registration *** Background Check**

🔍 **Q14. Check driver BG check status**

Find Turn.io BG check state for a driver

```
// Find by driver_id  
db.driver_registration_record.findOne(  
  { driver_id: 7873 }  
);
```

```
// Find by Turn reference_id  
db.driver_registration_record.findOne(  
  { reference_id: "abc123" }  
);
```

💡 *Key fields: turn_id (exo_id), worker_id (exo_transaction_id), reference_id, state. See Training Doc 11.5.*

Dimension Measurements

🔍 **Q15. Verify dimension measurement for a shipment**

After measuring a shipment via Warehouse App, verify it was recorded

```
db.measurement_obligations.findOne(
  { shipment_id: "SHIP12345" }
);
```

💡 *If returns null, measurement wasn't saved. Re-measure or check enable_measure config (Q13).*

Common Pitfalls **&**** Tips**

PostgreSQL – DBeaver

- If you don't see tables: enable 'Show all databases' in connection settings, OR specify the correct DB name (e.g. 'axlehire_dataorch') in the Database field.
- Use LIMIT in your queries – production tables can have millions of rows.
- Prefix all queries with SELECT first to verify data before running UPDATE/DELETE.

MongoDB

- Use .pretty() in Mongo shell to format output: db.collection.find({}).pretty()
- Compass GUI is easier than shell for new members – install it.
- Collections are case-sensitive: 'sort_center' is different from 'Sort_Center'.

StrongDM connection

- If query times out: check SDM client – resource may have disconnected. Click again to reconnect.
- Port changes per resource – always re-check in SDM when reconnecting.
- If 'Connection refused' – DevOps may have revoked your resource access. Contact #devops.

| **⚠️ Production warning** All queries above default to Staging. When you switch to Production resource in StrongDM, BE VERY CAREFUL. Always SELECT before UPDATE/DELETE. Always use a Test_ prefix for any data you create. See Training Doc Section 10 – Production Rules. |

| --- |

End of Database Cheat Sheet 📄

Always SELECT before UPDATE/DELETE. Always cleanup after testing.

Page of